

Document Scanner & Enhancer

Computer Vision — Course Project Report

Topic #16 (Documents): “Detect a document, correct its perspective, and improve readability.”

Team	Role
Volha Platnitskaya	Image Processing & Morphology (Enhance, Segment, Clean)
Evgeniya Yankovich	Lead CV Engineer (Detect, Decide, Integration)

Pipeline: image → enhance → segment → clean → detect → decide

Test set: three photographs of real paper documents and generated one on wooden/textured surfaces (an invoice, a DMV change-of-address form, a page from a church constitution, and a printed ML Kit document), each shot at an angle, with natural lighting, glare, and some back-side show-through.

1. Problem Description

Photographing a paper document with a phone almost never gives a clean, front-facing page. The sheet ends up rotated, stretched by perspective (a trapezoid instead of a rectangle), unevenly lit, and surrounded by whatever happens to be on the table. The goal of this project was to build a Computer Vision system that takes a photo like that and turns it into a flat, upright, readable scan — essentially the same job a mobile scanning app does.

Given an input photo, the system needed to:

- Locate the document and its four corners in the scene;
- Correct the perspective so the page becomes a flat, front-facing rectangle;
- Improve the readability of the corrected page so it looks scanner-like;
- Produce an automatic, interpretable decision — was a document found and scanned successfully or not.

The whole thing had to run automatically, with no manual clicks or corner adjustments anywhere in the core logic, and it had to work on at least three real test images while saving every intermediate stage, so the process stays transparent and easy to check.

2. Team Roles and Task Division

The project follows the five-stage pipeline required for the assignment, and each of us owned a set of stages end-to-end — writing the code, testing it, and being ready to explain it.

Member	Role	Pipeline stages / files
Volha Platnitskaya	Image Processing & Morphology	enhance.py, segment.py, clean.py, report.pdf
Evgeniya Yankovich	Lead CV Engineer (integration & QA)	detect.py, decide.py, pipeline.py, requirements.txt, report.pdf

The Problem description, Pipeline design, Conclusion and Contribution statement sections were written together. Volha wrote up the Enhance/Segment/Clean methods, and Evgeniya wrote the Detect/Decide methods, the combined Results section, and the failure-case analysis.

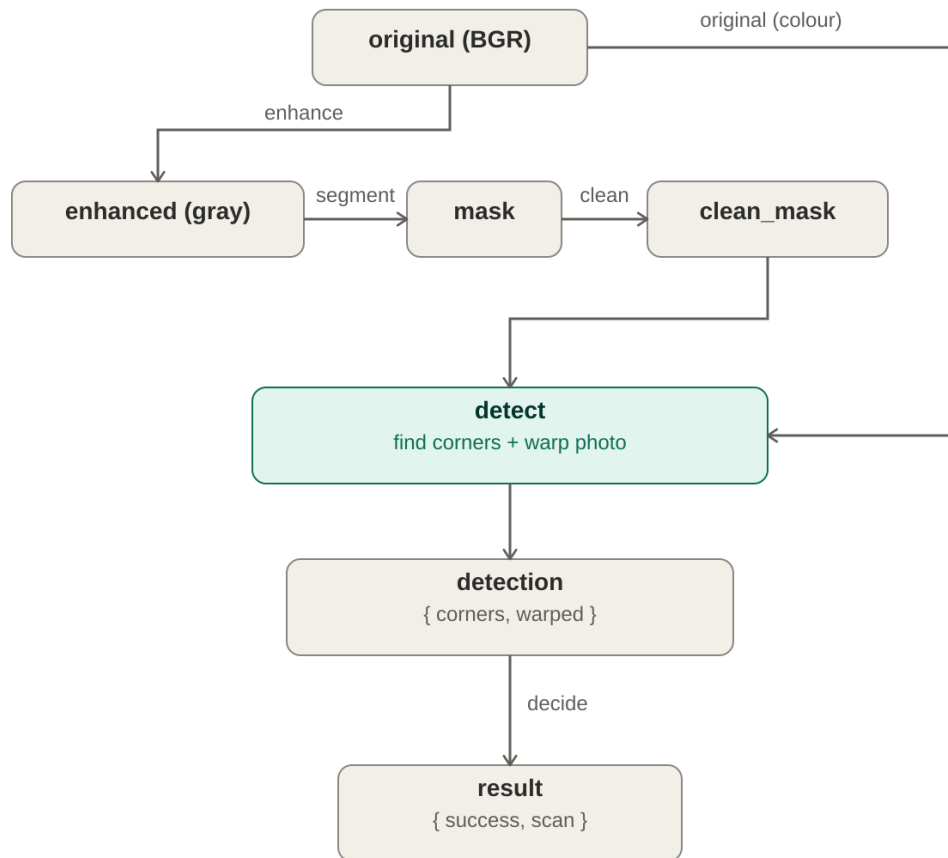
One decision we made early on, during integration, mattered a lot for how the rest of the pipeline turned out: since the assignment is about detecting the document boundary, the region of interest coming out of Segment/Clean had to be the silhouette of the sheet of paper itself — not a map of the text on it. We kept each stage doing only its own job (thresholding in Segment, morphology in Clean, contours/perspective in Detect) without any overlap between them.

3. Pipeline Design

The system is organized as five independent stages, all connected through a single driver script, pipeline.py. Each stage is a pure function with one clear entry point, which made it easy to keep our contributions separate and to test each piece on its own:

```
enhance(image)          -> grayscale, illumination-normalised image
segment(enhanced)       -> binary mask (document vs background)
clean(mask)             -> single solid document silhouette
detect(clean_mask, image) -> dict: found / corners / warped /
                             visualization
decide(detection)       -> dict: success / corners / scan / message
```

For a single photo, the data flows like this:



Note that Detect takes both the cleaned mask, to find the document, and the original colour image, so the perspective warp is applied to the full-quality photo rather than to the binary mask. pipeline.py runs this over every image in data/ and writes a complete, numbered record of the run into output/<name>/:

01_original.jpg	- input photo
02_enhanced.jpg	- after Enhance
03_mask.jpg	- segmentation mask
04_clean.jpg	- cleaned document silhouette
05_detection.jpg	- detected corners drawn on the original
06_warped.jpg	- perspective-corrected original (colour)
07_scan.jpg	- final readable scanner-style output
decision.txt	- automatic decision (message, success, corner count)

4. Methods Used

4.1 Enhance — Volha Platnitskaya

Purpose: normalise the illumination and cut down noise before any real analysis starts.

- Gamma correction ($\gamma = 1.2$) applied through a look-up table (cv2.LUT) to lift the mid-tones and even out the exposure.
- Conversion to grayscale — the geometry of the page doesn't depend on colour, and working in grayscale simplifies everything downstream.
- CLAHE (clipLimit = 2.0, tileGridSize = 8×8) to boost local contrast between the bright paper and the darker background without amplifying noise too much.
- Bilateral filtering (d = 7, sigmaColor = 50, sigmaSpace = 50) to smooth the surface texture while keeping edges sharp.

4.2 Segment — Volha Platnitskaya

Purpose: separate the document from the background.

- Gaussian blur (7×7) to suppress fine texture so the threshold step is stable.
- Otsu global thresholding (cv2.threshold with THRESH_BINARY + THRESH_OTSU), which automatically finds the split between the bright sheet and the darker surface and produces a filled silhouette of the document.

4.3 Clean — Volha Platnitskaya

Purpose: turn the raw mask into one solid, clean region.

- Connected-components analysis (cv2.connectedComponentsWithStats), keeping only the largest component — this removes most of the fragmented background texture in one step.
- Morphological opening (erosion then dilation), with a kernel size that scales with the image (about 2.5% of the shorter side), to cut thin “bridges” of glare that can connect the sheet to a bright patch of background.
- A final largest-component pass, just to guarantee we end up with exactly one solid silhouette.

4.4 Detect — Evgeniya Yankovich

Purpose: find the document and correct its perspective.

- Contour extraction (cv2.findContours, RETR_EXTERNAL) on the clean silhouette; the largest contour above 10% of the image area is taken to be the document.

- Quadrilateral fitting with `cv2.approxPolyDP`, increasing epsilon until we get a 4-corner convex polygon. If the contour is too irregular it falls back to `cv2.convexHull` + `approx`, and then to `cv2.minAreaRect`, so a four-corner result is always produced one way or another.
- Corner ordering — the four points are sorted into top-left, top-right, bottom-right, bottom-left using coordinate sums and differences.
- Perspective correction — the output rectangle size is derived from the real edge lengths of the detected quadrilateral, so the aspect ratio is preserved, and then `cv2.getPerspectiveTransform` + `cv2.warpPerspective` straighten the page.
- Visualization — the ordered corners and the detected quadrilateral are drawn on the original photo so the result can be checked by eye.

4.5 Decide — Evgeniya Yankovich

Purpose: produce the automatic decision and the final readable output.

- Decision logic: if no document quadrilateral was found, the result is a structured “failure” (success = False, message = “Document not detected”). Otherwise the result reports success, the four corners, and the message “Document successfully scanned”.
- Readability enhancement — this is the “improve readability” part of the assignment, applied to the perspective-corrected page: median blur (denoise) → CLAHE (local contrast) → adaptive Gaussian thresholding (blockSize = 31, C = 15), to get a crisp black-and-white “scanner” page.

Additionally, we tried four different readability recipes on the warped pages and compared them: (A) CLAHE + sharpen + adaptive threshold; (B) denoise + CLAHE + adaptive threshold + despeckle; (C) bilateral + CLAHE + a larger-block adaptive threshold + opening; and (D) a softer grayscale CLAHE output. Recipe (B) gave the cleanest and most legible result, so that’s what we kept — sharpening before thresholding was dropped because it just amplified the noise.

5. Results — Stage by Stage

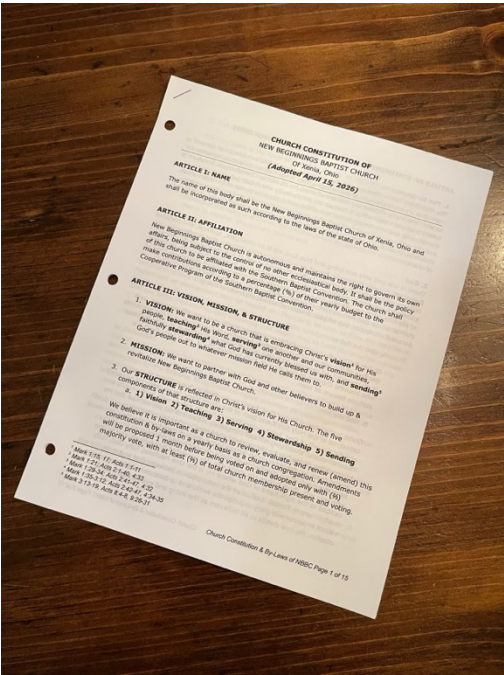
The pipeline was run on all four real document photos. Every one of them was detected and scanned successfully (decision.txt reported “Document successfully scanned” with 4 corners in each case).

Image	Document	Detection	Scan
test-image-1	Invoice (bamboo wood background)	OK (4 pts)	clean

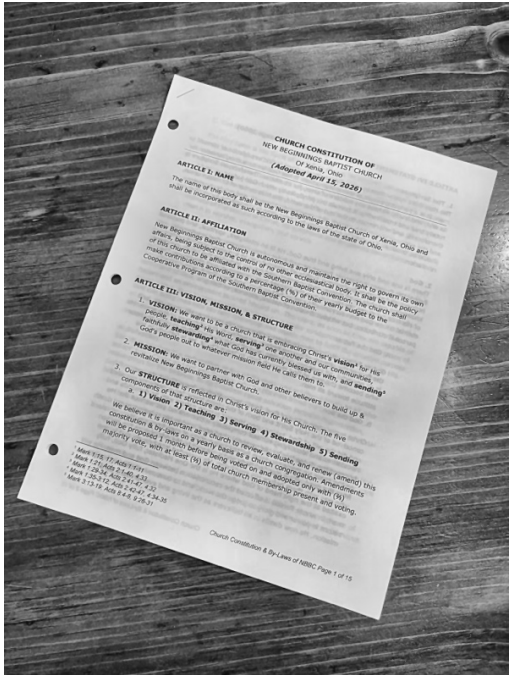
test-image-2	DMV change-of-address (dark wood)	OK (4 pts)	clean
test-image-3	Church constitution page (dark wood)	OK (4 pts)	clean
test-image-4	Printed ML Kit document (dark wood)	OK (4 pts)	clean

Below is the full stage-by-stage breakdown for test-image-3, since it's rotated and has glare, which makes it a good illustration of what each step is actually doing. Equivalent sets of images exist for test-image-1, test-image-2 and test-image-4 in the output folder.

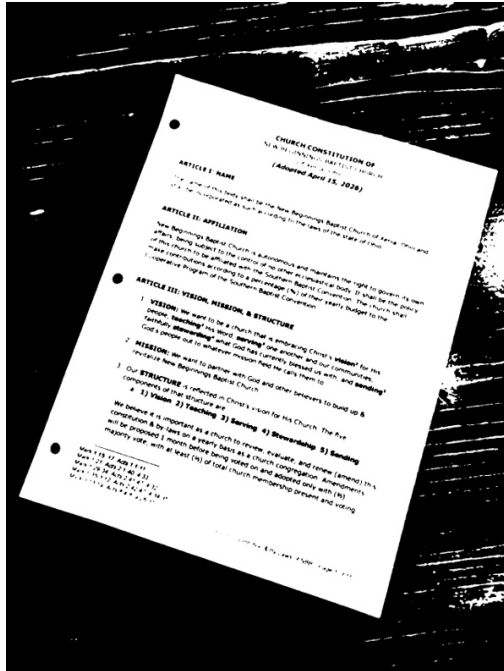
Original



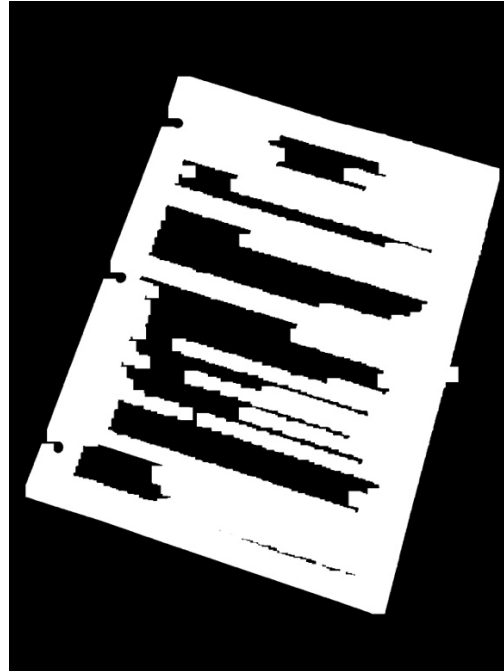
Enhance



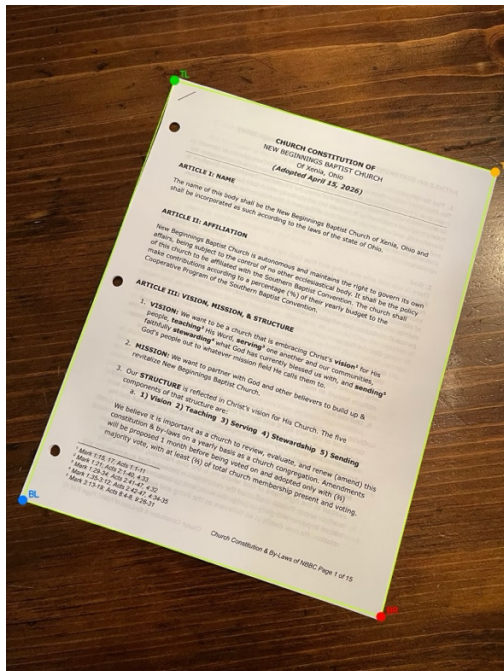
Segment



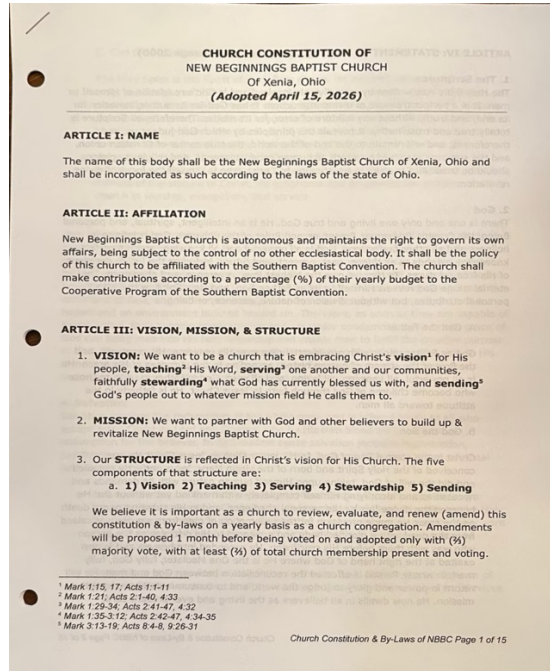
Clean



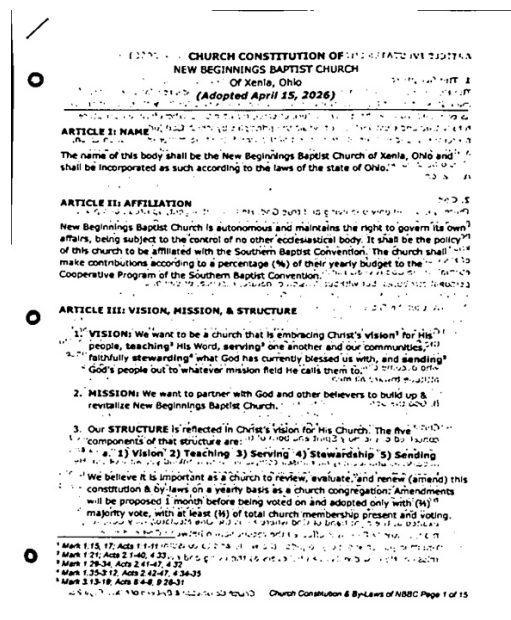
Detect



Warp



Final scan



Observations

- Corner detection holds up well even with about 15° of rotation and uneven lighting — all four corners land right on the page edges.
- The perspective warp produces an upright, correctly proportioned page, with the background removed entirely.
- The final scan is a high-contrast, readable black-and-white page that genuinely looks like it came out of a scanner; faint back-side show-through stays light while the real text comes out solid black.

6. Failure Cases

While building this we deliberately threw harder photos at the system to see where it breaks. Here's what we found:

1. Document not brighter than the background

Segment relies on Otsu thresholding, which assumes the sheet is the brighter, foreground object. On a white or very bright surface, or with a document darker than its background, that assumption flips and the silhouette collapses. A fix would be to add HSV/colour-based or adaptive segmentation, or detect the polarity automatically.

2. Folded or occluded corners

One of our early test images had a folded-over top-right corner. The detected corner got pulled toward the fold/background instead of the true paper corner, which distorted the warp. Heavily curled or partially hidden pages are still a hard case for us.

3. Strong glare bridging the page to the background

A bright reflection right next to the sheet can connect to the silhouette and merge with it. We mitigate this with a size-scaled morphological opening that cuts thin bridges, but a very large glare region could still merge with the document region.

4. Heavy back-side show-through

When text printed on the back of a thin sheet shows through, the adaptive threshold in the readability step picks it up as faint speckle. It stays lighter than the real text, so it doesn't ruin readability, but it's not fully removed either.

5. Multiple documents, or the page not being the largest bright object

Clean keeps only the single largest blob, so a scene with several sheets, or some other large bright object, can lead the system to pick the wrong region.

These cases line up with the assumptions we stated for the system: a single, mostly flat, bright document on a darker, contrasting background.

7. Conclusion

We ended up with a complete, fully automatic document-scanner pipeline that connects all five required CV stages and works end-to-end on real photos. Starting from a distorted phone photo, the system normalises illumination, segments the sheet with Otsu thresholding, cleans the mask into a solid silhouette using morphology and connected components, detects the four document corners, corrects the perspective, improves readability into a scanner-style page, and finally outputs an automatic, interpretable decision. Across our four-image test set, every document was detected and scanned correctly, and every stage is saved as a numbered image, so the whole process is easy to follow and check.

The biggest lesson for us was how much clean stage separation matters: choosing the document silhouette — not a text map — as the region of interest, and keeping thresholding strictly in Segment, morphology in Clean, and geometry in Detect, made the system both architecturally cleaner and measurably more robust. For further improvement, it is possible to add colour/adaptive segmentation to handle bright backgrounds, better handling of folded corners, and stronger show-through removal in the final scan.

Contribution Statement

Volha Platnitskaya — Image Processing & Morphology Specialist

Implemented and tested the Enhance (gamma, CLAHE, bilateral), Segment (Otsu thresholding) and Clean (connected components + morphological opening) stages, and wrote up their method descriptions, composed a project report.

Evgeniya Yankovich — Lead CV Engineer

Implemented and tested the Detect (contours, corner ordering, perspective warp) and Decide (decision logic + readability enhancement) stages; built pipeline.py and requirements.txt; integrated and verified the full system on the test set; wrote up the Results and the failure-case analysis, composed a project report.